Mise RMS: System Expansion & Architecture Roadmap

This document serves as the master engineering specification and architectural guideline for expanding the **Mise Restaurant Management System (RMS)** baseline.

---

1. Executive Summary & Purpose

Primary Objective

The objective of this specification is to govern the architectural growth of the Mise RMS system across human engineering teams and AI coding agents. Mise is designed to operate seamlessly under extreme network degradation and high-volume local operational stress typical of commercial food service environments.

Core Architecture Philosophy

- **Offline-First Authority:** The local SQLite instance is the single source of truth for immediate operational state. Cloud connectivity is an asynchronous secondary concern.
- **Deterministic Execution:** Financial, inventory, and audit transactions must yield deterministic outcomes regardless of device latency, network disconnection, or concurrent execution.
- **Agent Operational Guardrails:** AI agents implementing features in this repository must strictly adhere to the constraints defined in Section 6 to prevent architectural erosion, data corruption, or breaking local-first guarantees.

---

2. Module 1: Material & Grocery Inventory System

2.1 Recipe Bill of Materials (BOM) & Auto-Deduction

- **Sub-Unit Precision:** Inventory tracking must support high-precision decimal storage for quantities (grams, milliliters, unit counts) and integer storage for monetary values in base currency sub-units (e.g., paise).
- **Real-Time Deductions:** Execution of a Kitchen Order Ticket (KOT) triggers atomic triggers that reduce raw ingredient stock based on linked sub-recipe and master ingredient matrices.
- **Yield Factor Adjustments:** BOM calculations must factor in raw item yield percentages (e.g., meat trimming losses or vegetable peeling ratios) before subtracting net stock.

2.2 Purchase Orders (PO) & Vendor Directory

- **Par-Level Reordering:** Automatic generation of suggested POs when inventory levels cross configured safety thresholds ($Par_Level - Current_Stock + Lead_Time_Consumption$).
- **Vendor Management:** Maintain structured vendor directories with vendor-specific SKUs, minimum order quantities (MOQs), unit costs, and historical lead-time metrics.
- **Receiving & Cost Variance:** Support receiving workflows that allow itemized quantity checking and cost adjustments, updating the weighted average cost (WAC) per ingredient atomically.

2.3 Spoilage, Wastage & Non-Revenue Stock Tracking

- **Categorized Inventory Adjustments:** Track non-saleable stock movement using explicit classification categories:
    - Expired Perishables
    - Prep Line Spoilage / Burnt Food
    - Dropped / Damaged Items
    - Staff Meals & Promotional Samples
- **Variance Reconciliation:** Spoilage records must be isolated from unexplained inventory shrinkage during weekly physical stock counts to enable precise fraud detection.

2.4 FIFO & Batch Expiry Tracking

- **Lot-Based Ingestion:** Stock received via POs must be tagged with a Batch ID, Received Date, Expiry Date, and Initial Volume.
- **First-Expired, First-Out (FEFO) Enforcement:** Stock deduction algorithms must pull from the oldest non-expired batch.
- **Visual & System Alerts:**
    - **Amber Alert:** Batch within 48 hours of expiry (suggests daily specials usage).
    - **Red Alert:** Batch expired (hard-locks inventory item from automated KOT consumption).

---

3. Module 2: Enhanced Kitchen & Floor Operations

3.1 Course-Wise Fire Management

- **Logical Ordering Stages:** Split order items into distinct courses: *Appetizers (Course 1)*, *Mains (Course 2)*, *Desserts (Course 3)*, and *Beverages (Immediate)*.
- **KDS Hold & Release Flow:**
    - **Hold:** Items marked as held are visible on the Kitchen Display System (KDS) as greyed-out upcoming load indicators but do not trigger prep timers.
    - **Fire:** Manual or auto-timed release changes status to active, highlighting the item in green and initiating preparation SLA timers.

3.2 Table Operations: Splitting, Merging & Transfers

Operation	Trigger Condition	System Execution Rules
Split Table	Guest requests separate bills	Create child sub-orders linked to parent table. Re-allocate selected items while preserving overall KOT history.
Merge Tables	Large party joins adjacent tables	Combine target active session into destination table session. Retain item-level waitstaff metadata for tip distribution.
Tab Transfer	Bar guest moves to dining room	Migrate open tab items and pre-authorizations to dining table entity without closing the open order session.

3.3 Modifier & Allergen Hardening

- **Mandatory Modifier Rules:** Enforce required choices (e.g., Meat Temp: *Medium Rare*, Spice Level: *Medium*) prior to ticket submission. Order items cannot enter `PENDING_KOT` state without required parameters.
- **Allergen Alert Highlighting:** Critical allergen tags (*Peanut*, *Gluten*, *Shellfish*, *Dairy*) must render with high-contrast visual styling on print KOTs and physical KDS monitors, requiring explicit chef confirmation on screen.

---

4. Module 3: Advanced Billing, Tax & Financial Compliance

4.1 Multi-Jurisdiction Tax & Charge Engine

- **Hierarchical Tax Structure:** Flexible calculation engine capable of computing stacked tax models:
  - **Split Direct Taxes:** Simultaneous calculation of CGST (Central) + SGST (State) or IGST (Inter-state).
  - **Service Charges:** Configurable percentage-based or flat-rate service charges applied pre-tax or post-tax based on regional regulations.
  - **Packaging Fees:** Fixed or per-item fees automatically applied to takeaway and delivery order types.

4.2 Fraud Prevention & Security Audit Trails

```
None
[ Sensitive Operation Triggered ]
               |
               ▼
   Requires Manager Permission?
        ├── NO  ──► Execute Transaction & Log Standard Event
        └── YES ──► Prompt Manager Security PIN
                        |
                        ├── Invalid ──► Log Unauthorized Access Attempt &
Abort
                        └── Valid   ──► Log Supervisor ID + Reason Code ──►
Execute Action
```

- **Sensitive Operation Audits:** The following actions require an explicit Supervisor/Manager authorization PIN and append an immutable event to `audit_logs`:
  - Item voiding post-KOT printing.
  - Manual bill-level discounts exceeding authorized employee thresholds.
  - Manual cash drawer kick (no-sale open).
  - Reprinting finalized receipts.

4.3 Shift Close & Z-Report Cash Reconciliation

- **Blind Close Workflow:** Cashiers must input physical counts for all accepted payment tenders (Cash, Card, UPI, Vouchers) *before* the system reveals expected tallies.
- **Z-Report Generation:** Upon closing, the system locks all transactions for the session, calculates variances ($Physical_Count - System_Expected$), and mandates structured reason code entry for variances exceeding tolerance thresholds.

---

5. Module 4: Technical & Offline Architecture Enhancements

5.1 Local SQLite Offline Resilience & Sync Strategy

- **Optimistic Locking via Version Columns:** Every syncable entity must include an integer `version` field. Updates must fail and trigger reconciliation if the local version lags behind the write target:

$$UPDATE\ tables\ SET\ status = 'OCCUPIED',\ version = version + 1\ WHERE\ id = :id\ AND\ version = :expected_version;$$

- **Write-Ahead Logging (WAL):** SQLite databases on local nodes must operate in `journal_mode = WAL` with `synchronous = NORMAL` to maximize write throughput and minimize reader-writer lock contention during peak service hours.

5.2 Role-Based Access Control (RBAC) Matrix

Module / Action	Waitstaff	Kitchen Staff	Cashier	Manager
Create KOT / Add Items	**Yes**	No	**Yes**	**Yes**
Fire / Complete KDS Course	No	**Yes**	No	**Yes**
Print Final Bill	No	No	**Yes**	**Yes**
Post-KOT Void / Item Cancellation	Approval Required	No	Approval Required	**Yes**
Manage Inventory & POs	No	No	No	**Yes**
Run Z-Report Shift Close	No	No	**Yes**	**Yes**

---

6. AI Agent Implementation Guidelines & Non-Negotiable Guardrails

Mandatory Code & Architecture Standards

1. **Integer Currency Rule:** Never use floating-point types (`FLOAT`, `DOUBLE`) for monetary figures. All financial amounts must be computed and stored as integer sub-units (e.g., **Paise** for INR, **Cents** for USD).
2. **Immutable Bill Snapshots:** When an order is finalized and paid, serialize the item descriptions, prices, tax rates, and totals into an immutable JSON blob within the `finalized_bills` table. Future updates to menu pricing or tax configurations must never alter historical financial records.
3. **Local First, Zero Cloud Blocking:** Critical path operations (KOT creation, ordering, table management, bill printing) must execute synchronously against local SQLite tables. Cloud synchronization must occur asynchronously in background worker queues.
4. **Deterministic Schema Migrations:** Database schema modifications must be implemented via sequential, versioned SQL migration files (`V001__init.sql`, `V002__add_inventory.sql`). Dynamic, un-versioned schema alterations at application startup are strictly prohibited.

5. **Idempotent Sync Payloads:** All transaction sync payloads sent between local terminals and secondary devices must include a unique UUID `client_assigned_id` to prevent duplicate processing on retry operations.